

Decentralized Emergency Alert Network

Submitted by:

Aquitania, Earl Jasper

Corpuz, Kenneth Sean T.

Crema, Michaela Francine R.

Lopez, Jomel Dominic S.

Pumaras, Hurry T.

Introduction

This report documents the design and implementation of a Decentralized Emergency Alert Network — a multi-process, networked application built to simulate real-world fire and environmental hazard detection across a distributed environment. The system is composed of three cooperating components: a Naming Server, a Monitoring Server, and multiple Sensor Clients, all communicating over TCP sockets using structured JSON messages. The system models a realistic scenario in which geographically distributed sensors independently detect hazardous events (fire, smoke, or heat), report them to a central monitoring server, and collectively determine the causal order in which events occurred — even without access to a shared physical clock.

The following sections provide detailed explanations of how each architectural concern was addressed: how services discover one another at runtime through the Naming Server; how sensors and the monitoring server exchange structured JSON messages; and how Lamport Clocks enable meaningful event ordering across asynchronous processes. The report concludes with individual reflections from each team member on what the project taught them.

Naming

In a distributed system, processes cannot communicate unless they know where to find each other. Hardcoding IP addresses and ports is fragile and inflexible — it breaks the moment a service restarts on a different host. The Naming Server solves this problem by acting as a lightweight runtime directory: services register their addresses upon startup, and clients look up those addresses on demand.

How it was implemented:

The Naming Server (`naming_server.py`) is a standalone TCP server that maintains an in-memory registry mapping logical service names to (host, port) pairs. It listens for incoming connections and handles two types of messages:

- **register** — Sent by the Monitoring Server upon startup. The message contains the service's logical name (`monitoring.server.main`), its advertised IP address, and its port. The Naming Server stores this mapping and acknowledges with a success response.
- **lookup** — Sent by Sensor Clients before connecting to the Monitoring Server. The message contains the logical name to resolve. The Naming Server replies with the corresponding host and port, or an error if the name is not found.

The JSON message formats used are as follows:

Registration (Monitoring Server → Naming Server)

```
{ "type": "register", "name": "monitoring.server.main",  
  "host": "192.168.1.10", "port": 6060 }
```

Lookup (Sensor → Naming Server)

```
{ "type": "lookup", "name": "monitoring.server.main" }
```

Response (Naming Server → Sensor)

```
{ "status": "ok", "host": "192.168.1.10", "port": 6060 }
```

Each connection to the Naming Server is handled in its own thread (`handle_client`), allowing the server to serve multiple simultaneous requests without blocking. The registry is kept in a plain Python dictionary (`registry = {}`), which is sufficient for this prototype.

Configuration is fully environment-driven via `config.py`: the host and port of the Naming Server can be overridden through environment variables (`NAMING_SERVER_HOST`, `NAMING_SERVER_PORT`), making the system easily deployable across different machines without code changes. The Monitoring Server similarly uses `MONITORING_SERVER_ADVERTISE_HOST` to control what address it registers; decoupling the bind address (e.g., `0.0.0.0`) from the address clients should connect to.

This pattern mirrors real-world service registries such as Consul and Kubernetes DNS: services are loosely coupled through a central directory rather than hardcoded addresses, enabling flexible deployment and restart ability.

Message-Oriented Communication

All inter-process communication in the system is message-oriented: each interaction is represented as a discrete, self-describing JSON object transmitted over a TCP socket. This approach decouples the sender and receiver from each other's internal state — a process only needs to understand the message format, not the internals of the process that sent it.

How it was handled:

The system defines five message types, each with a defined sender, receiver, and structure:

Message Type	Sender	Receiver	Purpose
register	Monitoring Server	Naming Server	Advertise monitoring server address at startup
lookup	Sensor Client	Naming Server	Resolve the monitoring server address before connecting
alert	Sensor Client	Monitoring Server	Report a detected hazard event with Lamport timestamp
emergency_update	Monitoring Server	All Sensors	Broadcast confirmed fire status and causal ordering result

Alert Message Design

The alert message is the most information-rich message in the protocol. It carries:

- `sensor_id` — the identity of the reporting sensor
- `event` — the type of hazard detected (`fire_detected`, `smoke_detected`, `heat_detected`)
- `location` — the physical location string (e.g., Room 101)
- `timestamp` — the Lamport logical clock value at the time of event creation
- `detected_at_ns` — the wall-clock nanosecond timestamp captured at the moment of detection, before any simulated network delay

The separation between `timestamp` (Lamport) and `detected_at_ns` (wall clock) is intentional and critical. The Lamport timestamp is used for causal ordering; the nanosecond wall clock is used as a tiebreaker when two events share the same Lamport time. This dual-timestamp design ensures

that artificially delayed messages (simulated via `SENSOR_DELAY_BY_ID`) do not corrupt the causal ordering — because the Lamport stamp is assigned before the delay is applied.

Broadcast

The Monitoring Server maintains a list of all connected client sockets. When a confirmed fire alert is processed, it calls `broadcast()`, which iterates over all sockets and sends the `emergency_update` message to every connected sensor. Disconnected clients are detected by failed `send()` calls and pruned from the list automatically, keeping the system robust to sensor dropouts.

Lamport Clocks

In a distributed system, each process maintains its own independent clock. These clocks are not synchronized — they drift relative to one another, and network messages arrive with unpredictable delays. This makes it impossible to use wall-clock timestamps to determine the true order in which events occurred across processes.

In this project, the problem is made explicit through `SENSOR_DELAY_BY_ID` in `config.py`. `SensorB` is artificially delayed by 2 seconds, `SensorC` by 3 seconds, and so on. A sensor that detected an event earlier may send its alert message later due to this delay — causing naive wall-clock ordering to produce incorrect results.

Lamport Clock Algorithm

The `LamportClock` class (`lamport.py`) implements the standard Lamport clock algorithm:

- `send_event()` — increments the clock before sending a message, and returns the new value to be embedded in the message.
- `receive_event(received_time)` — updates the clock to $\max(\text{local}, \text{received}) + 1$ upon receiving a message, and returns the new value.
- `tick()` — increments the clock for a local event (not used in the current implementation but available).

How they are used in the system:

Each process maintains its own LamportClock instance:

- **Sensor Client:** calls `clock.send_event()` immediately when a hazard is detected. The returned value is embedded in the alert message as timestamp. This stamp is assigned before any artificial delay, preserving causal accuracy.
- **Monitoring Server:** calls `clock.receive_event(sensor_timestamp)` when an alert arrives. This advances the server's clock past the sender's, maintaining the happened-before relationship.
- **Sensor Client (on broadcast):** calls `clock.receive_event(message['lamport_time'])` when an `emergency_update` is received, keeping all clocks consistent.

Event Ordering

Upon receiving each alert, the Monitoring Server appends it to a shared alerts list and re-sorts the entire list using a three-level sort key:

- **Primary:** `sensor_timestamp` (Lamport time) — causal order

- Secondary: `detected_at_ns` (wall-clock nanoseconds) — tiebreaker for events with equal Lamport times
- Tertiary: `sensor_id` (string) — deterministic tiebreaker when both timestamps are equal

The first element of the sorted list is declared the first detector and is included in the `emergency_update` broadcast to all sensors. This ensures every node in the system agrees on the same causal ordering, which is the fundamental guarantee that Lamport Clocks provide.

Guarantee and Limitation

Lamport Clocks provide the happened-before guarantee: if event A causally precedes event B, then A's Lamport timestamp is strictly less than B's. However, they do not provide the converse: a lower timestamp does not imply causal precedence for concurrent events. For full causal tracking across concurrent branches, Vector Clocks would be required. For this project's use case — determining which sensor first reported a hazard in a single-server topology — Lamport Clocks are sufficient and appropriate.

Reflection

Each team member reflects below on their personal takeaways from the project, focusing on concepts learned, challenges encountered, and skills developed.

Member 1 – Hurry

Working on this project gave me a concrete understanding of how distributed systems manage service discovery at runtime. Before this, I thought of IP addresses as something you simply configure once. Implementing the Naming Server made me realize that in real systems, services come and go — they restart, move to new hosts, and scale horizontally. A naming layer decouples these dynamics from the rest of the system. I also learned how threading works in Python's socket model, and why each client connection needs its own thread to avoid blocking the main accept loop. The experience of debugging registration and lookup failures across processes was eye-opening — seeing how a single missed byte in a JSON message could cause a cryptic exception downstream taught me to be deliberate about protocol design.

Member 2 – Jomel

My biggest takeaway from this project is the importance of separating concerns when designing distributed protocols. The decision to use `detected_at_ns` alongside the Lamport timestamp was something I initially didn't understand — why capture wall time at all if we're using logical clocks? Working through the sorting logic in the Monitoring Server made it click: wall time gives us a physically meaningful tiebreaker for events that appear causally concurrent (same Lamport value), while the Lamport timestamp gives us causal correctness. I also came to appreciate how the artificial delays in `SENSOR_DELAY_BY_ID` turned out to be one of the most instructive parts

of the project. Without them, everything would arrive in roughly the right order by accident, and we'd never see the bugs that logical clocks are designed to fix.

Member 3 – Kenneth

I worked primarily on the sensor client side, which helped me understand the challenges of building a client in a system where the server's address is not known at compile time. The lookup flow — querying the naming server, receiving a host-port pair, then connecting to the monitoring server — felt natural once implemented, but required careful error handling at each step. I also learned about thread safety: the sensor client runs a background listener thread and a foreground send loop simultaneously, both sharing the same socket. I realized that without proper design, concurrent reads and writes on a socket can corrupt messages. In this implementation, reads and writes go in separate directions (server sends, client receives on the listener; client sends on the main thread), which avoids conflicts — a subtle but important design choice.

Member 4 – Michaella

Lamport Clocks were entirely new to me before this project. What struck me most is how elegant the algorithm is — it requires no external coordination, no shared memory, and no network round-trips to maintain a consistent logical ordering. Each process just increments a counter and piggybacks it on messages. Yet from this simple rule, you get a global ordering guarantee across arbitrarily many processes. Implementing the LamportClock class was straightforward, but seeing it actually used in context — where the monitoring server's clock advances each time it receives a message — made the abstraction come alive. I now understand why Lamport's 1978 paper is considered foundational: the idea is simple, but the insight behind it is profound. I would like to

explore Vector Clocks next, to understand how to track concurrent causality across multiple event chains.

Member 5 – Earl

This project was my first hands-on experience with socket programming and I found it more nuanced than I expected. Sockets are a low-level abstraction — you're responsible for framing your own messages, handling partial reads, and managing connection lifecycle. JSON over TCP worked well for our use case, but I noticed that our current `recv(1024)` pattern assumes each message fits within 1024 bytes. In a production system with larger payloads, this could silently truncate messages. I learned to think critically about the gap between 'works in testing' and 'works reliably in production.' I also appreciated working with the `config.py` environment variable pattern — it made deploying across different machines seamless without touching source code. Overall, this project gave me a strong foundation in network programming that I plan to build on.

Conclusion

The Distributed Sensor Alert System successfully demonstrates three core principles of distributed systems within a single cohesive project. The Naming Server provides runtime service discovery, decoupling sensor clients from hardcoded server addresses and enabling flexible, environment-driven deployment. Message-Oriented Communication through a well-defined JSON protocol enables clean, stateless interactions between all system components. Lamport Logical Clocks solve the event-ordering problem that physical clocks cannot address in the presence of asynchronous networks and artificial delays.

Together, these components form a functioning prototype that models real-world distributed architectures. The lessons learned — about protocol design, concurrency, logical time, and the gap between local correctness and distributed consistency — are broadly applicable to the design of production distributed systems.